

FitBuddy – AI Fitness Plan Generator using Gemini Models

Project Description:

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goals, such as weight loss, muscle gain, or general wellness. Built with FastAPI and integrated with Google's Gemini AI models, FitBuddy simplifies the process of creating structured and effective fitness routines.

The application takes in user information—like age, weight, goal, and preferred workout intensity—and generates a customized 7-day fitness plan. Users can also submit feedback to update and refine their plan using AI. A complementary nutrition or recovery tip is also provided based on their selected goal.

With a clean user interface and database support via SQLite, FitBuddy ensures each user's data and generated plan are stored and accessible. It offers both API endpoints and an interactive HTML interface for ease of use.

Scenarios:

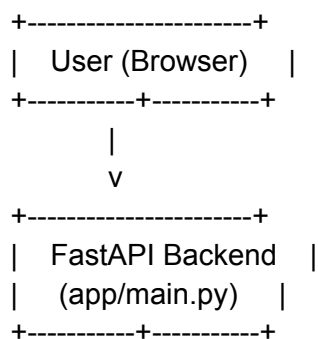
- * Scenario 1: A user inputs their personal details including name, age, weight, fitness goal (e.g., "weight loss" or "muscle gain"), and preferred workout intensity ("high", "medium" or "low") to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's fitness objective and experience level. This AI-driven customization enhances user motivation and adherence by providing structured and goal-specific routines.

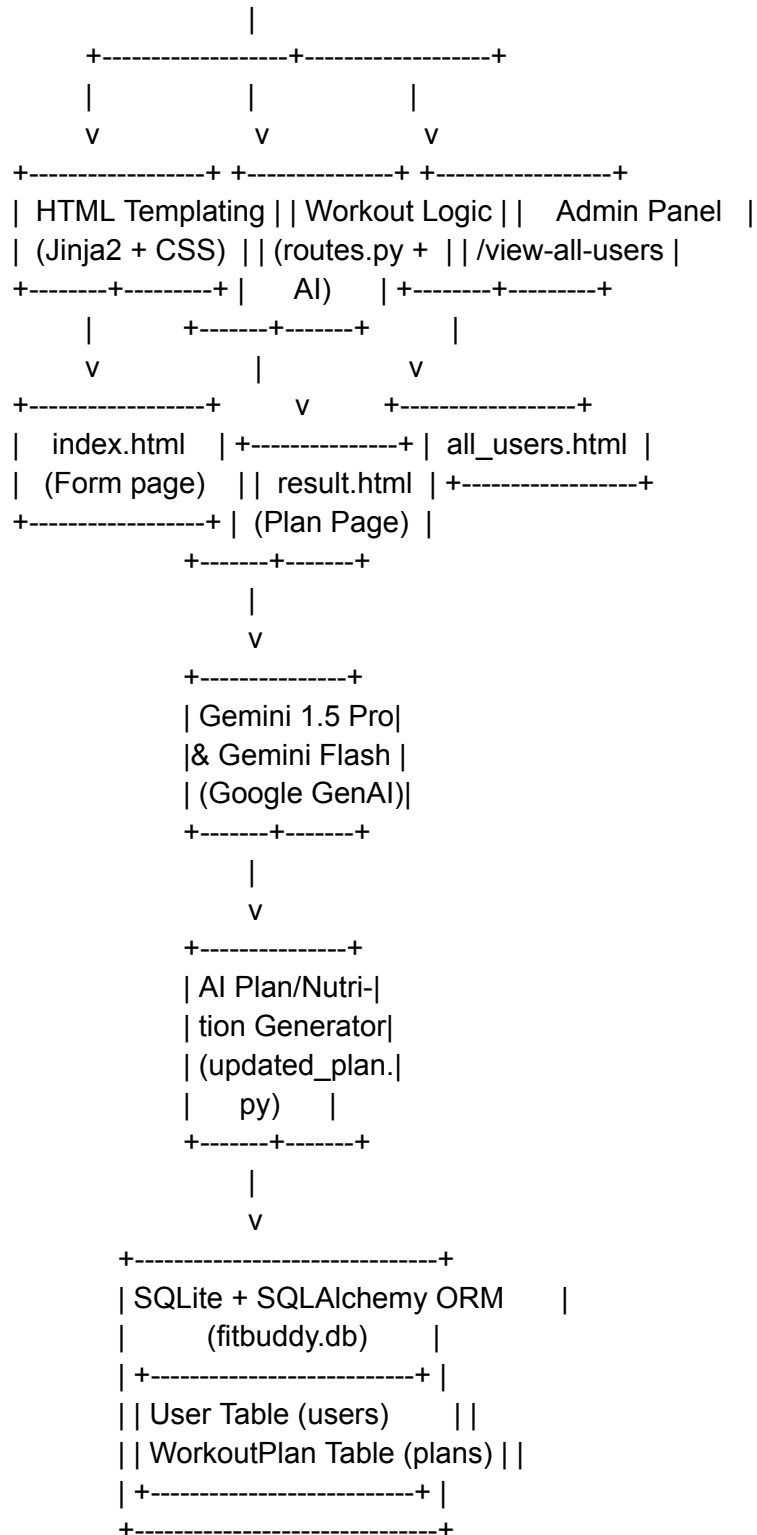
- * Scenario 2: A user submits feedback on their existing workout plan, such as suggesting changes like "more focus on cardio" or "include more rest days." The system uses this input to regenerate an updated workout plan, refining it based on the user's evolving needs. This feedback loop ensures that users stay engaged and receive dynamic fitness support as their preferences or progress change.

- * Scenario 3: A user requests a nutrition or recovery tip based on their selected fitness goal. The platform delivers a concise and relevant suggestion (e.g., "Include protein in your post-workout meal" for muscle gain), which complements the workout plan. These targeted tips help users improve results by aligning their diet and recovery practices with their physical training regimen.

- * Scenario 4: An admin or coach accesses the platform to view all registered users and their workout plans through a dedicated dashboard. This view allows them to track individual progress, compare original and updated plans, and monitor user activity. It streamlines oversight for trainers managing multiple clients or students.

Technical Architecture:





Pre-requisites:

- * FastAPI Framework Knowledge: [FastAPI Documentation](#)
- * Gemini API Familiarity: [Google Generative AI Documentation](#)

- * HTML, CSS, and Jinja2 Template Skills: W3Schools HTML/CSS/Jinja2 Tutorials
- * Python Programming Proficiency: Python Official Docs
- * Version Control with Git: Git Documentation
- * SQLAlchemy and SQLite Basics: SQLAlchemy Docs
- * Environment Setup (pip, virtualenv, or conda): Virtualenv Guide
- * Uvicorn ASGI Server: Uvicorn Docs

Project Workflow:

Milestone 1. Model selection and Architecture

- * Activity 1.1. Research and Select the Appropriate Generative AI Model
- * Activity 1.2. Define the architecture of the application
- * Activity 1.3. Set up the development Environment

Milestone 2. Core functionalities Development

- * Activity 2.1. Develop the core functionalities
- * Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3. route.py Development

- * Activity 3.1. Writing the Main Application Logic in routes.py

Milestone 4. Frontend Development

- * Activity 4.1. Designing and Developing User Interface
- * Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2

Milestone 5. Deployment

- * Activity 5.1. Preparing the Application for Local Deployment
- * Activity 5.2. Testing and Verifying Local Deployment

Milestone Details

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google's Gemini family for the FitBuddy application. This includes evaluating the performance and capabilities of multiple Gemini models to ensure they align with the application's core functionalities—such as generating personalized workout plans, updating them based on feedback, and providing tailored nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy's AI model needed to:

- * Generate structured 7-day workout plans
- * Provide concise and actionable nutrition tips
- * Update workout plans based on user feedback
- * Perform well in real-time, web-based interaction

Model Evaluation

We initially experimented with several Hugging Face models:

- * T5 & BART – For text generation (workout/nutrition)
- * DistilBERT – For potential intent classification
- * Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

These models required:

- * Hosting with Hugging Face or local GPU setup

- * Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

- * Gemini 1.5 Pro
 - * Used for structured 7-day workout generation and plan updating
 - * Delivers rich, context-aware, formatted responses reliably
- * Gemini Flash
 - * Used for generating fast, practical nutrition tips
 - * Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

- * API-based, easy to integrate with FastAPI
- * Naturally understands structured formatting (days, sections)
- * High accuracy with minimal prompt tuning
- * Scalable and fast for feedback-based revisions

Activity 1.2. Define the architecture of the application

Architecture Overview: FitBuddy follows a modular FastAPI-based architecture—

- * Frontend: HTML + Jinja2 templates for user input and display
- * Backend: FastAPI for route handling and logic
- * AI Layer: Google Gemini APIs (Pro & Flash)
- * Database: SQLite with SQLAlchemy for persistence

Frontend Responsibilities: Users interact via—

- * index.html – Input form
- * result.html – Plan, tip, feedback
- * all_users.html – Admin panel

Backend Responsibilities:

- * Receives form data
- * Calls Gemini models
- * Stores user info and plans

Here is the complete transcription of the newly uploaded document images:

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

- * Developed index.html as the main entry point for user input.
- * Structured the form to capture:
 - * username, user_id, age, weight, goal, and intensity.
- * Each field is properly labeled and grouped using semantic HTML.
- * The design uses a gym-themed background image, Google Fonts (Roboto), and bold typography to create a clean, fitness-oriented interface.
- * Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

- * Embedded CSS styles were applied directly in each HTML file.
- * Layouts use Flexbox to center content both vertically and horizontally.
- * Media queries ensure mobile responsiveness.
- * Buttons have consistent styling with hover effects.

- * Input fields and result blocks have adequate padding, shadows, and rounded corners for modern aesthetics.

- * Overall color scheme supports readability on a dark background.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under /templates:

- * index.html: Collects user inputs
- * result.html: Displays the generated AI plan
- * all_users.html: Lists all user records for admin view

Milestone 5. Deployment

Now, the focus is on deploying the FitBuddy application on a local system using FastAPI and Uvicorn. The app uses HTML templates rendered with Jinja2, alongside AI services powered by Gemini. This milestone ensures all environment dependencies are correctly installed, and the app runs smoothly on a local server, preparing it for future deployment on cloud platforms.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

- * Create a virtual environment to manage dependencies and isolate the FitBuddy project from other environments.
- * Activate the environment and install all necessary dependencies listed in the requirements.txt file, including FastAPI, Uvicorn, SQLAlchemy, and Google Generative AI SDK.

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Configure environment variables

- * Use environment variables to store sensitive data such as the Gemini API Key.
- * These variables can be set directly in the terminal or added to a .env file which is loaded into the application using python-dotenv.

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via Jinja2-rendered templates:

- * Workout Intensity (Low, Medium, High)

Once users submit this data, they can click the "Generate Plan" button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The visual appeal of the background image featuring an active individual reinforces the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

Activity 5.2: Testing and Verifying Local Deployment

Access the Application Locally

The FastAPI development server must be started using the command (uvicorn main:app --reload). Once the server is running, the application can be accessed through any web browser by navigating to http://127.0.0.1:8000. This URL loads the main user

interface where individuals can input their details and generate personalized fitness plans. Additionally, the interactive API documentation provided by FastAPI can be viewed at [\[http://127.0.0.1:8000/docs\]](http://127.0.0.1:8000/docs)(<http://127.0.0.1:8000/docs>), allowing developers to explore and test available endpoints.

Exploring Website's web pages:

Home page:

- > FitBuddy - AI Workout Generator
- > Form Fields: Name, User ID, Age, Weight (kg), Fitness Goal, Workout Intensity.
- > Button: Generate Plan
- >

Personalized workout generation page:

- > 🏋️ Your Personalized Workout Plan
- > Displays user information alongside day-by-day workout details, tips, and the option to submit feedback.
- >

Conclusion:

The FitBuddy - AI Fitness Plan Generator is a comprehensive, user-centric web application that showcases the powerful synergy between artificial intelligence and health-tech solutions. Designed to assist individuals in achieving their fitness goals, FitBuddy uses Google's Gemini 1.5 Pro and Gemini Flash models to generate personalized 7-day workout plans and daily nutrition or recovery tips based on user inputs such as age, weight, fitness goal, and preferred workout intensity.

The application is built using the FastAPI framework for backend logic and API handling, SQLAlchemy with SQLite for persistent data storage, and Jinja2 templating for a clean and responsive frontend. Upon submitting their details on the homepage, users receive a tailored workout plan that includes warm-ups, main workouts, cooldowns, and goal-aligned tips. What sets FitBuddy apart is its dynamic feedback feature—users can provide input on the effectiveness or difficulty of their current plan, and the system responds by intelligently updating the plan using generative AI, ensuring ongoing customization and engagement.

Additionally, the application includes an administrative view where all registered users and their respective plans (original and updated) are displayed in a structured format. This functionality supports transparency, monitoring, and further optimization, making FitBuddy suitable for both individual users and coaches or institutions managing multiple fitness profiles.

Overall, FitBuddy stands as a scalable and adaptable AI-powered solution that not only personalizes the fitness journey but also evolves with the user's progress and preferences. It demonstrates how cutting-edge technology can be integrated into everyday wellness routines to promote healthy, consistent, and goal-oriented lifestyles.

Based on the uploaded document images, here are the transcriptions of all distinct sections visible across the pages:

Milestone 3. routes.py Development

Milestone 3 focuses on building the routing logic inside the routes.py file, which acts as the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (Gemini Pro and Flash), and the SQLite database. Each route defines how user inputs

are collected, processed, and responded to — creating a dynamic, personalized experience for fitness planning and feedback-based updates.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py:

Separate routes were implemented to serve each of FitBuddy's main functionalities:

- * / : Displays the user form via index.html
- * /generate-workout: Processes user input and generates a personalized workout plan
- * /submit-feedback: Updates the workout plan based on user feedback
- * /view-all-users: Displays an admin dashboard showing all users and their plans

Set up Route Handlers for Each Feature

► / - Home Route

- * Returns the homepage (index.html)
- * No form processing logic

► /generate-workout - Plan Generator

- * Receives user input using FastAPI's Form(...)

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in routes.py. Each route is linked to the corresponding function developed in Activity 2.1.

Process User Input

- * HTML Form: Created in index.html
- * Users enter: username, user_id, age, weight, goal, and intensity
- * FastAPI captures inputs using Form(...) parameters
- * Pydantic schemas (UserInput, FeedbackRequest) are used to validate structure

Integrate Gemini API Calls

Each route handler connects directly to the respective Gemini-powered function:

- * /generate-workout \rightarrow
 - * generate_workout_gemini() for workout plan
 - * generate_nutrition_tip_with_flash() for tip
- * /submit-feedback \rightarrow
 - * Retrieves plan
 - * Sends original + feedback to update_workout_plan()

User & Plan Storage

Functions:

- * save_user(), save_plan(), update_plan()
- * get_original_plan(), get_user()

File: database.py

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved as a separate field to preserve both versions (original and updated).

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):  
    db = SessionLocal()  
    existing = db.query(User).filter_by(id=user_id).first()  
    if existing:
```

```

        # Update existing user info
        existing.name = name
        existing.age = age
        existing.weight = weight
        existing.goal = goal
        existing.intensity = intensity
    else:
        # Create a new user
        user = User(
            id=user_id,
            name=name,
            age=age,
            weight=weight,
            goal=goal,
            intensity=intensity,
            schedules=[] # default schedule or logic
        )
        db.add(user)
    db.commit()
    db.close()

# stores the plan in the database
def save_plan(user_id: int, plan: str):
    db = SessionLocal()
    workout = WorkoutPlan(user_id=user_id, original_plan=plan)
    db.add(workout)
    db.commit()
    db.close()

def get_user(user_id: int):
    db = SessionLocal()
    return db.query(User).filter(User.id == user_id).first()

```

Admin View of All Users

- * Route: /view-all-users
- * Template: all_users.html

Displays a list of all users and their respective plans. Rendered with Jinja2 using a {% for user in users %} loop, it allows administrators to see original and updated plans clearly in a structured table format.

Milestone 1. Model Selection and Architecture

In this milestone, the focus is on selecting the appropriate generative AI models from Google's Gemini family for the FitBuddy application. This includes evaluating the performance and capabilities of multiple Gemini models to ensure they align with the application's core

functionalities—such as generating personalized workout plans, updating them based on feedback, and providing tailored nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy's AI model needed to:

- * Generate structured 7-day workout plans
- * Provide concise and actionable nutrition tips
- * Update workout plans based on user feedback
- * Perform well in real-time, web-based interaction

Model Evaluation

We initially experimented with several Hugging Face models:

- * T5 & BART - For text generation (workout/nutrition)
- * DistilBERT - For potential intent classification

These models required:

- * Hosting with Hugging Face or local GPU setup
- * Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

- * Gemini 1.5 Pro
 - * Used for structured 7-day workout generation and plan updating
 - * Delivers rich, context-aware, formatted responses reliably
- * Gemini Flash
 - * Used for generating fast, practical nutrition tips
 - * Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

- * API-based, easy to integrate with FastAPI
- * Naturally understands structured formatting (days, sections)
- * High accuracy with minimal prompt tuning
- * Scalable and fast for feedback-based revisions

Activity 1.2: Define the Architecture of the Application

Architecture Overview:

FitBuddy follows a modular FastAPI-based architecture:

- * Frontend: HTML + Jinja2 templates for user input and display

Set Up Project Structure:

Create folders: /app (Python files), /templates (HTML), /static/images (assets)

fitbuddy/

```
|
|— requirements.txt      # Project dependencies
|
|— app/
|   |— main.py          # FastAPI entry point
|   |— routes.py        # Core route handlers
|   |— database.py      # SQLAlchemy models and DB logic
|   |— schemas.py       # Pydantic models for validation
```

```

├── gemini_generator.py # Gemini Pro - workout plan generator
├── gemini_flash_generator.py # Gemini Flash - nutrition tips
├── updated_plan.py     # Feedback-based plan updater
├── nutrition.py        # Handles nutrition-specific logic (optional)
├── templates/
│   ├── index.html      # User input form
│   ├── result.html     # Workout plan, tip, feedback
│   └── all_users.html   # Admin dashboard
├── static/
│   └── images/
│       └── gym-bg.jpg   # Gym-themed background
└── fitbuddy.db          # Local SQLite database (auto-generated)

```

Milestone 2. Core Functionalities Development

Activity 2.1 Develop the Core Functionalities:

Workout Plan Generation

* Function: `generate_workout_gemini()`

* File: `gemini_generator.py`

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to the Gemini 1.5 Pro model, which returns a structured, day-wise plan including:

* Warm-up (5-10 mins)

* Main workout (exercise details, sets, reps)

* Cooldown or recovery tip

The output is returned to `routes.py` and rendered on the `result.html` page using `<pre>` blocks for formatting.

Setup and Running Instructions

Create Virtual Environment:

```
python -m venv fitbuddy-env
```

```
fitbuddy-env\Scripts\activate (Windows)
```

Install Required Libraries:

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

Run Server:

```
uvicorn app.main:app --reload
```

Visit <http://127.0.0.1:8000> for the app and `/docs` for API testing.

Based on the newly provided document images, here is the transcription of the additional sections:

Project Scenarios

Scenario 1: A user inputs their personal details including name, age, weight, fitness goal (e.g., "weight loss" or "muscle gain"), and preferred workout intensity ("high", "medium", or "low") to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's fitness objective and experience level. This AI-driven customization enhances user motivation and adherence by providing structured and goal-specific routines.

Scenario 2: A user submits feedback on their existing workout plan, such as suggesting changes like "more focus on cardio" or "include more rest days." The system uses this input to regenerate an updated workout plan, refining it based on the user's evolving needs. This feedback loop ensures that users stay engaged and receive dynamic fitness support as their preferences or progress change.

Scenario 3: A user requests a nutrition or recovery tip based on their selected fitness goal. The platform delivers a concise and relevant suggestion (e.g., "Include protein in your post-workout meal" for muscle gain), which complements the workout plan. These targeted tips help users improve results by aligning their diet and recovery practices with their physical training regimen.

Scenario 4: An admin or coach accesses the platform to view all registered users and their workout plans through a dedicated dashboard. This view allows them to track individual progress, compare original and updated plans, and monitor user activity. It streamlines oversight for trainers managing multiple clients or students.

Prerequisites & Project Workflow

Pre-requisites:

- * FastAPI Framework Knowledge: [FastAPI Documentation](#)
- * Gemini API Familiarity: [Google Generative AI Documentation](#)
- * HTML, CSS, and Jinja2 Template Skills: [W3Schools HTML/CSS/Jinja2 Tutorials](#)
- * Python Programming Proficiency: [Python Official Docs](#)
- * Version Control with Git: [Git Documentation](#)
- * SQLAlchemy and SQLite Basics: [SQLAlchemy Docs](#)
- * Environment Setup (pip, virtualenv, or conda): [Virtualenv Guide](#)
- * Uvicorn ASGI Server: [Uvicorn Docs](#)

Project Outline / Index

- * Milestone 1. Model selection and Architecture
 - * Activity 1.1. Research and Select the Appropriate Generative AI Model
 - * Activity 1.2. Define the architecture of the application
 - * Activity 1.3. Set up the development Environment
- * Milestone 2. Core functionalities Development
 - * Activity 2.1. Develop the core functionalities
 - * Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing
- * Milestone 3. route.py Development
 - * Activity 3.1. Writing the Main Application Logic in routes.py
- * Milestone 4. Frontend Development
 - * Activity 4.1. Designing and Developing User Interface
 - * Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2
- * Milestone 5. Deployment
 - * Activity 5.1. Preparing the Application for Local Deployment

* Activity 5.2. Testing and Verifying Local Deployment

Additional Details on Model Selection & Architecture

- * Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

Frontend Responsibilities: Users interact via–

- * index.html - Input form
- * result.html - Plan, tip, feedback
- * all_users.html - Admin panel

Additional Core Functionality Implementation Details

Nutrition Tip Generation

- * Function: generate_nutrition_tip_with_flash()
- * File: gemini_flash_generator.py

This function generates a concise, practical nutrition or recovery tip tailored to the user's selected goal. It utilizes the Gemini Flash model to deliver a fast, helpful response such as hydration advice, protein suggestions, or general recovery best practices. This tip is displayed beneath the workout plan in result.html.

Feedback-Based Plan Updating

- * Function: update_workout_plan()
- * File: updated_plan.py

This function enables the user to revise their original plan by submitting feedback (e.g., "Add yoga", "Include more cardio"). The original plan and feedback are passed to Gemini 1.5 Pro, which returns a modified plan with the suggested updates. The updated plan is then stored and displayed alongside the new nutrition tip.

```
def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    """Uses Gemini 1.5 Pro to update the workout plan based on user feedback."""
    prompt = f"""
    You are a professional fitness trainer assistant.
```

Here's the original 7-day workout plan:

{original_plan}

User Feedback:

"{user_feedback}"

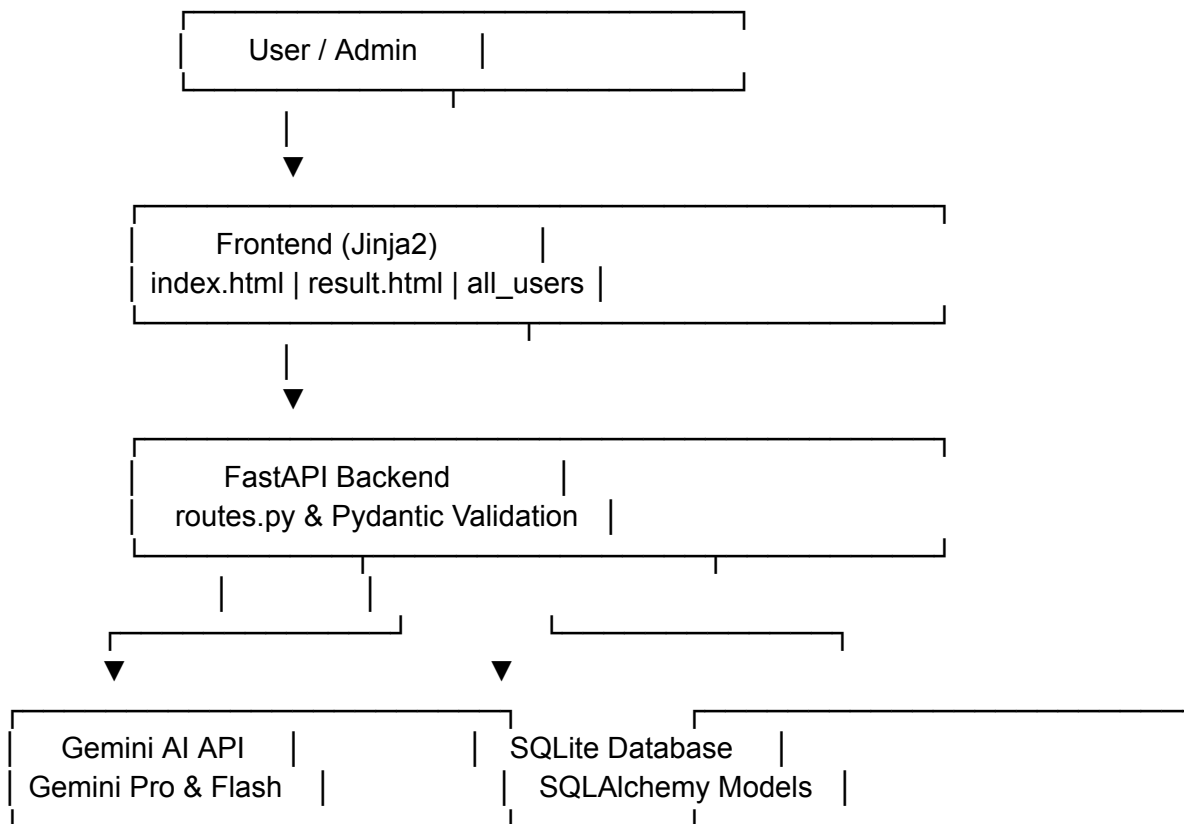
Based on the feedback, revise the relevant parts of the workout plan. Keep the format and rest of the plan unchanged if not needed.

```
"""
try:
    response = model.generate_content(prompt)
    return response.text.strip()
except Exception as e:
    return f"Error updating plan: {str(e)}"
```

Project Overview & Objectives

FitBuddy is a web-based AI fitness application built with FastAPI and integrated with Google Gemini AI models. It provides personalized 7-day fitness plans, dynamic workout updates based on user feedback, nutrition tips, and an admin interface to monitor user data using SQLite and SQLAlchemy.

Core Components & System Flow



Implementation Breakdown

1. Development Environment Setup

To initialize the project environment, run the following commands:

Create and activate virtual environment

```
python -m venv fitbuddy-env
```

```
fitbuddy-env\Scripts\activate # On Windows
```

Install required packages

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

Run the local server

```
uvicorn app.main:app --reload
```

Access the app at [\[http://127.0.0.1:8000\]](http://127.0.0.1:8000)<http://127.0.0.1:8000> and API documentation at [\[http://127.0.0.1:8000/docs\]](http://127.0.0.1:8000/docs)<http://127.0.0.1:8000/docs>.

2. AI Plan & Tip Generation Logic (`gemini_generator.py`)

This component interacts directly with the Gemini API to handle plan generation and nutrition advice:

```
import google.generativeai as genai
```

```
# Configure API Key
```

```
genai.configure(api_key="YOUR_GEMINI_API_KEY")
```

```
# Initialize Models
```

```
pro_model = genai.GenerativeModel("gemini-1.5-pro")
```

```
flash_model = genai.GenerativeModel("gemini-1.5-flash")
```

```
def generate_workout_gemini(user_input: dict) -> str:
```

```
    """Generates a structured 7-day workout plan using Gemini 1.5 Pro."""
```

```
    prompt = f"""
```

```
    You are a professional fitness trainer.
```

```
    Create a personalized 7-day workout plan for a user with the following details:
```

```
    - Age: {user_input.get('age')}
```

```
    - Weight: {user_input.get('weight')} kg
```

```
    - Fitness Goal: {user_input.get('goal')}
```

```
    - Intensity: {user_input.get('intensity')}
```

```
    For each day, include:
```

```
    1. Warm-up (5-10 mins)
```

```
    2. Main workout (exercise details, sets, reps)
```

```
    3. Cooldown or recovery tip
```

```
    """
```

```
    try:
```

```
        response = pro_model.generate_content(prompt)
```

```
        return response.text.strip()
```

```
    except Exception as e:
```

```
        return f"Error generating workout plan: {str(e)}"
```

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
```

```
    """Generates a quick nutrition or recovery tip using Gemini Flash."""
```

```
    prompt = f"Provide a short, practical nutrition or recovery tip for someone whose goal is '{goal}'."
```

```
    try:
```

```
        response = flash_model.generate_content(prompt)
```

```
        return response.text.strip()
```

```
    except Exception as e:
```

```
        return f"Error generating nutrition tip: {str(e)}"
```

3. Plan Revisions via User Feedback (updated_plan.py)

This component updates an existing plan based on specific user feedback:

```

def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    """Uses Gemini 1.5 Pro to update the workout plan based on user feedback."""
    prompt = f"""
    You are a professional fitness trainer assistant.

    Here's the original 7-day workout plan:
    {original_plan}

    User Feedback:
    "{user_feedback}"

    Based on the feedback, revise the relevant parts of the workout plan.
    Keep the format and rest of the plan unchanged if not needed.
    """
    try:
        response = pro_model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error updating plan: {str(e)}"

```

4. Database Models & Storage (database.py)

Handles persistence for user records, original plans, and feedback updates using SQLAlchemy:

```

from sqlalchemy import create_engine, Column, Integer, String, Float, Text
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

```

```

DATABASE_URL = "sqlite:///./fitbuddy.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

```

```

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, unique=True, index=True)
    name = Column(String)
    age = Column(Integer)
    weight = Column(Float)
    goal = Column(String)
    intensity = Column(String)
    original_plan = Column(Text, nullable=True)
    updated_plan = Column(Text, nullable=True)
    nutrition_tip = Column(Text, nullable=True)

```

```
Base.metadata.create_all(bind=engine)
```

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
```

```
    """Saves or updates user metadata in SQLite."""
```

```
    db = SessionLocal()
```

```
    existing = db.query(User).filter_by(user_id=user_id).first()
```

```
    if existing:
```

```
        existing.name = name
```

```
        existing.age = age
```

```
        existing.weight = weight
```

```
        existing.goal = goal
```

```
        existing.intensity = intensity
```

```
    else:
```

```
        new_user = User(
```

```
            user_id=user_id, name=name, age=age,
```

```
            weight=weight, goal=goal, intensity=intensity
```

```
        )
```

```
        db.add(new_user)
```

```
    db.commit()
```

```
    db.close()
```

```
def save_plan(user_id: int, original_plan: str, tip: str):
```

```
    """Saves the initial AI-generated plan and nutrition tip."""
```

```
    db = SessionLocal()
```

```
    user = db.query(User).filter_by(user_id=user_id).first()
```

```
    if user:
```

```
        user.original_plan = original_plan
```

```
        user.nutrition_tip = tip
```

```
        db.commit()
```

```
    db.close()
```

```
def update_plan_db(user_id: int, revised_plan: str):
```

```
    """Saves the user-revised workout plan."""
```

```
    db = SessionLocal()
```

```
    user = db.query(User).filter_by(user_id=user_id).first()
```

```
    if user:
```

```
        user.updated_plan = revised_plan
```

```
        db.commit()
```

```
    db.close()
```

```
def get_original_plan(user_id: int) -> str:
```

```
    """Retrieves original workout plan for a user."""
```

```
    db = SessionLocal()
```

```

user = db.query(User).filter_by(user_id=user_id).first()
plan = user.original_plan if user else ""
db.close()
return plan

```

```

def get_all_users():
    """Retrieves all registered user records for admin display."""
    db = SessionLocal()
    users = db.query(User).all()
    db.close()
    return users

```

5. Routing & Integration (routes.py)

Connects endpoints with Pydantic validations, Gemini logic, and Jinja2 template rendering:

```

from fastapi import APIRouter, Request, Form, Depends
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
from gemini_generator import generate_workout_gemini, generate_nutrition_tip_with_flash
from updated_plan import update_workout_plan
import database as db

```

```

router = APIRouter()
templates = Jinja2Templates(directory="templates")

```

Pydantic Schemas for validation

```

class UserInput(BaseModel):
    username: str
    user_id: int
    age: int
    weight: float
    goal: str
    intensity: str

```

```

@router.get("/", response_class=HTMLResponse)
def read_root(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

```

```

@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(
    request: Request,
    username: str = Form(...),
    user_id: int = Form(...),
    age: int = Form(...),

```

```

weight: float = Form(...),
goal: str = Form(...),
intensity: str = Form(...)
):
    user_data = {
        "username": username, "user_id": user_id, "age": age,
        "weight": weight, "goal": goal, "intensity": intensity
    }

    # Save user details
    db.save_user(user_id, username, age, weight, goal, intensity)

    # Generate workout plan and nutrition tip
    plan = generate_workout_gemini(user_data)
    tip = generate_nutrition_tip_with_flash(goal)

    # Store initial plan
    db.save_plan(user_id, plan, tip)

    return templates.TemplateResponse("result.html", {
        "request": request,
        "username": username,
        "user_id": user_id,
        "workout_plan": plan,
        "nutrition_tip": tip
    })

@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(
    request: Request,
    user_id: int = Form(...),
    user_feedback: str = Form(...)
):
    original_plan = db.get_original_plan(user_id)
    revised_plan = update_workout_plan(original_plan, user_feedback)

    # Update revised plan in database
    db.update_plan_db(user_id, revised_plan)

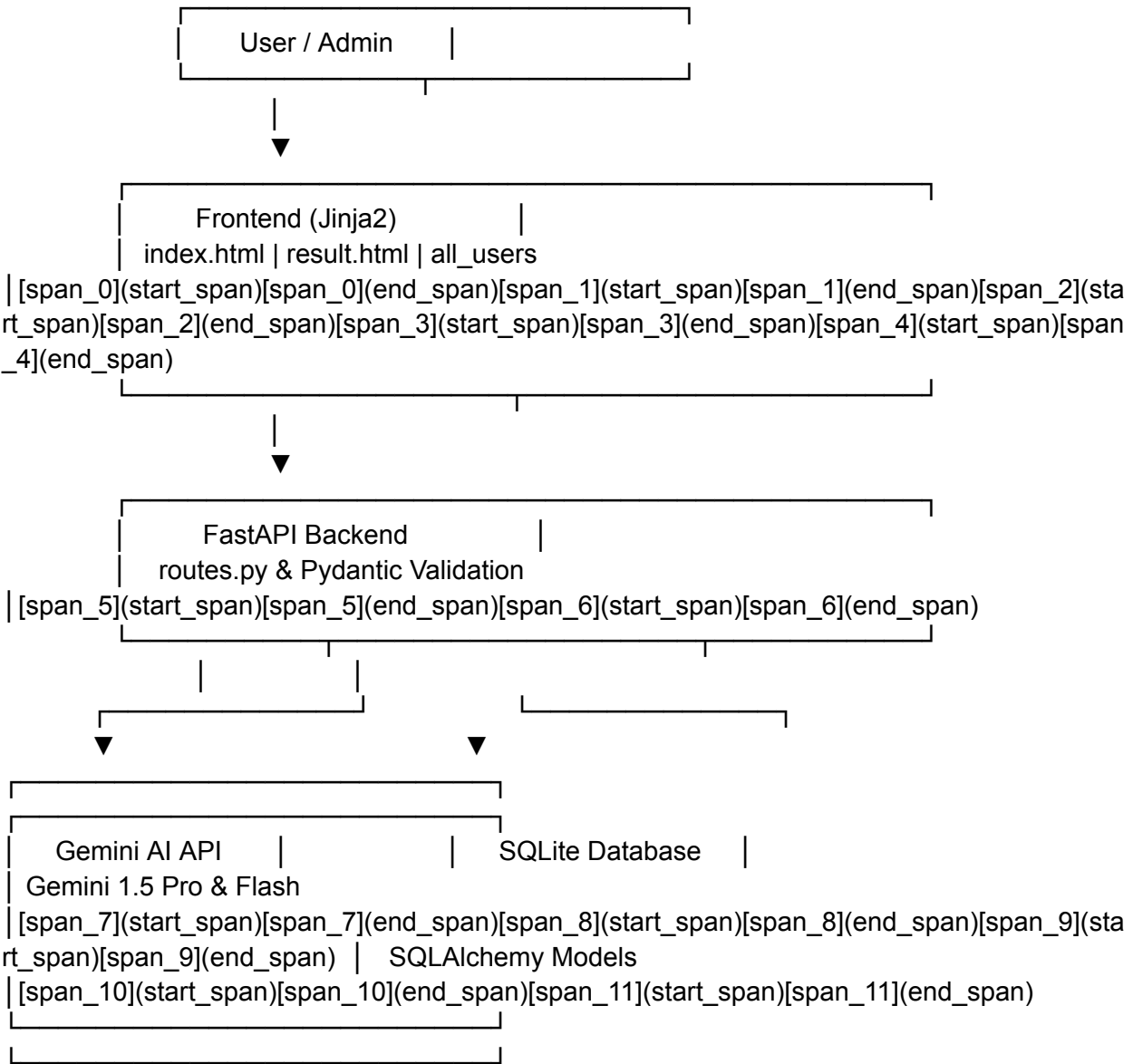
    return templates.TemplateResponse("result.html", {
        "request": request,
        "user_id": user_id,
        "workout_plan": revised_plan,
        "feedback_applied": True
    })

```

```
}}
```

```
@router.get("/all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    users = db.get_all_users()
    return templates.TemplateResponse("all_users.html", {
        "request": request,
        "users": users
    })
```

Technical System Flow & Architecture



Implementation Breakdown

1. Gemini AI Integration (gemini_generator.py & updated_plan.py)

Handles generating personalized workout plans, nutrition tips, and revising existing plans based on user feedback.

```
import google.generativeai as genai
```

```
# Configure API Key
```

```
genai.configure(api_key="YOUR_GEMINI_API_KEY")
```

```
# Initialize Gemini Models
```

```
pro_model = genai.GenerativeModel("gemini-1.5-pro")
```

```
flash_model = genai.GenerativeModel("gemini-1.5-flash")
```

```
def generate_workout_gemini(user_input: dict) -> str:
```

```
    """Generates a structured 7-day workout plan using Gemini 1.5
```

```
    Pro."""[span_15](start_span)[span_16](start_span)"[span_15](end_span)[span_16](end_span)
```

```
    prompt = f"""
```

```
    You are a professional fitness trainer.
```

```
    Create a personalized 7-day workout plan based on these user details:
```

```
    - Age: {user_input.get('age')}
```

```
    - Weight: {user_input.get('weight')} kg
```

```
    - Fitness Goal: {user_input.get('goal')}
```

```
    - Intensity Level: {user_input.get('intensity')}
```

```
    Include for each day: Warm-up, Main Workout (exercises, sets, reps), and Cooldown.
```

```
    """
```

```
    try:
```

```
        response = pro_model.generate_content(prompt)
```

```
        return response.text.strip()
```

```
    except Exception as e:
```

```
        return f"Error generating workout plan: {str(e)}"
```

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
```

```
    """Generates a quick nutrition or recovery tip using Gemini 1.5
```

```
    Flash."""[span_17](start_span)[span_18](start_span)"[span_17](end_span)[span_18](end_span)
```

```
    prompt = f"Provide a concise, actionable nutrition and recovery tip for a fitness goal of:
```

```
    {goal}."
```

```
    try:
```

```
        response = flash_model.generate_content(prompt)
```

```
        return response.text.strip()
```

```
    except Exception as e:
```

```
        return f"Error generating nutrition tip: {str(e)}"
```

```
def update_workout_plan(original_plan: str, user_feedback: str) -> str:
```

```
"""Revises an existing workout plan using Gemini 1.5 Pro based on user
feedback."""[span_19](start_span)[span_20](start_span)"[span_19](end_span)[span_20](end_sp
an)
```

```
prompt = f"""
```

```
You are an AI fitness coach.
```

```
Original Workout Plan:
```

```
{original_plan}
```

```
User Feedback:
```

```
"{user_feedback}"
```

Revise the relevant parts of the workout plan based on the feedback while keeping the overall format consistent.

```
"""
```

```
try:
```

```
    response = pro_model.generate_content(prompt)
```

```
    return response.text.strip()
```

```
except Exception as e:
```

```
    return f"Error updating workout plan: {str(e)}"
```

2. Database Layer (database.py)

Manages persistence for users, original plans, revised plans, and admin analytics using SQLAlchemy and SQLite.

```
from sqlalchemy import create_engine, Column, Integer, String, Float, Text
```

```
from sqlalchemy.ext.declarative import declarative_base
```

```
from sqlalchemy.orm import sessionmaker
```

```
DATABASE_URL = "sqlite:///./fitbuddy.db"
```

```
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```
class User(Base):
```

```
    __tablename__ = "users"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    user_id = Column(Integer, unique=True, index=True)
```

```
    name = Column(String)
```

```
    age = Column(Integer)
```

```
    weight = Column(Float)
```

```
    goal = Column(String)
```

```
    intensity = Column(String)
```

```
    original_plan = Column(Text, nullable=True)
```

```
updated_plan = Column(Text, nullable=True)
nutrition_tip = Column(Text, nullable=True)
```

```
Base.metadata.create_all(bind=engine)
```

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
    """Saves user information to
    SQLite."""[span_23](start_span)[span_24](start_span)"[span_23](end_span)[span_24](end_span
    )
    db = SessionLocal()
    user = db.query(User).filter_by(user_id=user_id).first()
    if user:
        user.name, user.age, user.weight, user.goal, user.intensity = name, age, weight, goal,
intensity
    else:
        user = User(user_id=user_id, name=name, age=age, weight=weight, goal=goal,
intensity=intensity)
        db.add(user)
        db.commit()
        db.close()
```

```
def save_plan(user_id: int, original_plan: str, tip: str):
    """Stores generated plan and nutrition
    tip."""[span_25](start_span)[span_26](start_span)"[span_25](end_span)[span_26](end_span)
    db = SessionLocal()
    user = db.query(User).filter_by(user_id=user_id).first()
    if user:
        user.original_plan = original_plan
        user.nutrition_tip = tip
        db.commit()
    db.close()
```

```
def update_plan(user_id: int, revised_plan: str):
    """Updates user plan in database after feedback
    processing."""[span_27](start_span)[span_28](start_span)"[span_27](end_span)[span_28](end_s
pan)
    db = SessionLocal()
    user = db.query(User).filter_by(user_id=user_id).first()
    if user:
        user.updated_plan = revised_plan
        db.commit()
    db.close()
```

```
def get_original_plan(user_id: int) -> str:
```

```

    """Retrieves stored original plan for feedback
revision."""[span_29](start_span)[span_30](start_span)"[span_29](end_span)[span_30](end_spa
n)
    db = SessionLocal()
    user = db.query(User).filter_by(user_id=user_id).first()
    plan = user.original_plan if user else ""
    db.close()
    return plan

```

```

def get_all_users():
    """Fetches all users and plans for the admin
view."""[span_31](start_span)[span_32](start_span)"[span_31](end_span)[span_32](end_span)
    db = SessionLocal()
    users = db.query(User).all()
    db.close()
    return users

```

3. FastAPI Core Routes (routes.py)

Configures route handlers, Pydantic validation schemas, and template rendering logic.

```

import os
from fastapi import APIRouter, Request, Form
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
from gemini_generator import generate_workout_gemini, generate_nutrition_tip_with_flash
from updated_plan import update_workout_plan
import database as db

```

```

router = APIRouter()

```

```

# Template directory setup

```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")
templates =
Jinja2Templates(directory=TEMPLATE_DIR)[span_36](start_span)[span_36](end_span)

```

```

# Pydantic Schemas for structural validation

```

```

class
UserInput(BaseModel):[span_37](start_span)[span_37](end_span)[span_38](start_span)[span_
38](end_span)
    username: str
    user_id: int
    age: int
    weight: float

```

```
goal: str
intensity: str
```

```
class FeedbackRequest(BaseModel):[span_39](start_span)[span_39](end_span)
```

```
    user_id: int
    user_feedback: str
```

```
# 1. Home Route
```

```
@router.get("/",
response_class=HTMLResponse)[span_40](start_span)[span_40](end_span)[span_41](start_sp
an)[span_41](end_span)
def read_root(request: Request):
    """Displays the user input form via
index.html."""[span_42](start_span)[span_43](start_span)[span_44](start_span)"[span_42](end_s
pan)[span_43](end_span)[span_44](end_span)
    return templates.TemplateResponse("index.html", {"request": request})
```

```
# 2. Plan Generator Route
```

```
@router.post("/generate-workout",
response_class=HTMLResponse)[span_45](start_span)[span_45](end_span)[span_46](start_sp
an)[span_46](end_span)[span_47](start_span)[span_47](end_span)
def generate_workout(
    request: Request,
    username: str =
Form(...),[span_48](start_span)[span_48](end_span)[span_49](start_span)[span_49](end_span)
[span_50](start_span)[span_50](end_span)
    user_id: int =
Form(...),[span_51](start_span)[span_51](end_span)[span_52](start_span)[span_52](end_span)
    age: int =
Form(...),[span_53](start_span)[span_53](end_span)[span_54](start_span)[span_54](end_span)
    weight: float =
Form(...),[span_55](start_span)[span_55](end_span)[span_56](start_span)[span_56](end_span)
    goal: str =
Form(...),[span_57](start_span)[span_57](end_span)[span_58](start_span)[span_58](end_span)
    intensity: str =
Form(...)[span_59](start_span)[span_59](end_span)[span_60](start_span)[span_60](end_span)
):
    """Processes user input, calls Gemini, and renders
result.html."""[span_61](start_span)[span_62](start_span)[span_63](start_span)[span_64](start_s
pan)"[span_61](end_span)[span_62](end_span)[span_63](end_span)[span_64](end_span)
    user_data = {
        "username": username, "user_id": user_id, "age": age,
        "weight": weight, "goal": goal, "intensity": intensity
    }
```

```

# Save input and execute AI tasks
db.save_user(user_id, username, age, weight, goal,
intensity)[span_65](start_span)[span_65](end_span)[span_66](start_span)[span_66](end_span)
workout_plan =
generate_workout_gemini(user_data)[span_67](start_span)[span_67](end_span)[span_68](start
_span)[span_68](end_span)[span_69](start_span)[span_69](end_span)
nutrition_tip =
generate_nutrition_tip_with_flash(goal)[span_70](start_span)[span_70](end_span)[span_71](sta
rt_span)[span_71](end_span)[span_72](start_span)[span_72](end_span)

db.save_plan(user_id, workout_plan,
nutrition_tip)[span_73](start_span)[span_73](end_span)[span_74](start_span)[span_74](end_sp
an)

return templates.TemplateResponse("result.html",
{[span_75](start_span)[span_75](end_span)[span_76](start_span)[span_76](end_span)[span_7
7](start_span)[span_77](end_span)
    "request": request,
    "username": username,
    "user_id": user_id,
    "age": age,
    "weight": weight,
    "goal": goal,
    "intensity": intensity,
    "workout_plan": workout_plan,
    "nutrition_tip": nutrition_tip
})[span_78](start_span)[span_78](end_span)

```

3. Feedback Revision Route

```

@router.post("/submit-feedback",
response_class=HTMLResponse)[span_79](start_span)[span_79](end_span)[span_80](start_sp
an)[span_80](end_span)[span_81](start_span)[span_81](end_span)[span_82](start_span)[span
_82](end_span)
def submit_feedback(
    request: Request,
    user_id: int =
Form(...)[span_83](start_span)[span_83](end_span)[span_84](start_span)[span_84](end_span)
    user_feedback: str =
Form(...)[span_85](start_span)[span_85](end_span)[span_86](start_span)[span_86](end_span)
):
    """Retrieves original plan, processes feedback with Gemini, and updates
DB."""[span_87](start_span)[span_88](start_span)[span_89](start_span)[span_90](start_span)"[s
pan_87](end_span)[span_88](end_span)[span_89](end_span)[span_90](end_span)

```

```

original_plan =
db.get_original_plan(user_id)[span_91](start_span)[span_91](end_span)[span_92](start_span)[
span_92](end_span)[span_93](start_span)[span_93](end_span)
revised_plan = update_workout_plan(original_plan,
user_feedback)[span_94](start_span)[span_94](end_span)[span_95](start_span)[span_95](end
_span)[span_96](start_span)[span_96](end_span)

db.update_plan(user_id,
revised_plan)[span_97](start_span)[span_97](end_span)[span_98](start_span)[span_98](end_s
pan)

return templates.TemplateResponse("result.html",
{[span_99](start_span)[span_99](end_span)[span_100](start_span)[span_100](end_span)
"request": request,
"user_id": user_id,
"workout_plan": revised_plan,
"feedback_applied": True
})

```

4. Admin Dashboard Route

```

@router.get("/view-all-users",
response_class=HTMLResponse)[span_101](start_span)[span_101](end_span)[span_102](star
t_span)[span_102](end_span)
def view_all_users(request: Request):
    """Displays user details and their plans on
all_users.html."""[span_103](start_span)[span_104](start_span)[span_105](start_span)"[span_10
3](end_span)[span_104](end_span)[span_105](end_span)
    users = db.get_all_users()[span_106](start_span)[span_106](end_span)
    return templates.TemplateResponse("all_users.html",
{[span_107](start_span)[span_107](end_span)[span_108](start_span)[span_108](end_span)
"request": request,
"users": users
})

```

4. Frontend Templates & Render Engine

Main Application Entry point (templates/index.html)

Captures user attributes using standard semantic HTML input elements formatted with CSS

Flexbox layout:

```

<form action="/generate-workout" method="post">
  <input type="text" name="username" placeholder="Username" required>
  <input type="number" name="user_id" placeholder="User ID" required>
  <input type="number" name="age" placeholder="Age" required>
  <input type="number" step="0.1" name="weight" placeholder="Weight (kg)" required>
  <input type="text" name="goal" placeholder="Fitness Goal" required>

```

```

    <input type="text" name="intensity" placeholder="Intensity Level" required>
    <button type="submit">Generate Plan</button>
</form>

```

Admin Records View (templates/all_users.html)

Iterates through database user objects using Jinja2 syntax to display saved workout plans cleanly inside <pre> blocks:

```

<table>
  <tr>

<th>ID</th><th>Username</th><th>Age</th><th>Weight</th><th>Goal</th><th>Intensity</th>
<th>Original Plan</th><th>Updated Plan</th>
  </tr>
  {% for user in users %}
  <tr>
    <td>{{ user.user_id }}</td>
    <td>{{ user.name }}</td>
    <td>{{ user.age }}</td>
    <td>{{ user.weight }}</td>
    <td>{{ user.goal }}</td>
    <td>{{ user.intensity }}</td>
    <td><pre>{{ user.original_plan }}</pre></td>
    <td><pre>{{ user.updated_plan }}</pre></td>
  </tr>
  {% endfor %}
</table>

```

5. Local Execution Instructions

Execute the application environment locally using standard uvicorn web server commands:

Setup environment

```
python -m venv venv
```

```
venv\Scripts\activate
```

Install dependencies

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

Boot local Uvicorn server

```
uvicorn main:app --reload
```